

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Project Overview

This is a **vision-based autonomous smart car** project for the 21st National Undergraduate Smart Car Competition (全国大学生智能车大赛). The car uses camera-based lane detection with PID control for autonomous navigation across various road elements (crossings, ramps, circles, forks, obstacles, bridges, zebra crossings).

Workspace Structure

The VS Code workspace (`DZXS_CYT4BB.code-workspace`) spans three MCU platform projects:

Directory	MCU	IDE	Purpose
<code>../../New_path/IAR/CYT4BB7_Library-master(2)/Seekfree_CYT4BB_Opensource_Library</code>	Infineon CYT4BB7 (Cortex-M7 dual-core)	IAR EW for Arm ≥9.40.1	Active development target
<code>../../Control/Seekfree_TC264_Opensource_Library.zip</code>	Infineon TC264D (TriCore)	TASKING/AURIX IDE	Primary working implementation (currently archived as zip; ref/TC264 is empty)
<code>../ref/RT1064_SJTU</code>	NXP RT1064 (Cortex-M7)	MDK/Keil	Reference implementation (SJTU AuTop team) — visual code base for CYT4BB7 port

Application code lives in `project/code/` and `project/user/` (CYT4BB7), or `Project/CODE/` and `Project/USER/src/` (RT1064).

Note: The `vision/` root itself contains **no active `code/` or `user/` files**. Active firmware is in the external directories above. `vision/` holds tools, docs, and historical reference code.

Building

There is **no command-line build system**. Each platform requires its vendor IDE:

- TC264:** Open in TASKING/AURIX Development Studio → Build project. Output: `.elf` / `.hex` in `Debug/` directory.
- CYT4BB7:** Open `project/iar/cyt4bb7.eww` in IAR Embedded Workbench for ARM (≥9.40.1 recommended) → Build. Clean (`project→clean`) is required after moving the project.
- RT1064:** Open `Project/MDK/RT106X.uvprojx` in Keil MDK → Build.

Debugging uses CMSIS-DAP ARM debugger.

Architecture

Data Flow

MT9V03X Camera (grayscale) → DMA/CSI into RAM buffer →
(CYT4BB7: CM7_0 acquires → CM7_1 processes via shared RAM)
(RT1064: CSI double-buffer → RT-Thread main task)
Image Processing (Adaptive threshold / Find line / IPM) →
Boundary Detection → Center Line Calculation → Road Type Classification →
PID Control (speed + steering + gyro) → Motor PWM Output

Image Processing Pipeline

Aspect	CYT4BB7	RT1064 (Reference)	TC264 (Prior knowledge)
Native resolution	MT9V03X_W=188 , MT9V03X_H=120	MT9V03X_CSI_W=376 , MT9V03X_CSI_H=240 (CSI); or 188×120 (FlexIO)	MT9V03X_W=188 , MT9V03X_H=120 typical
Processed resolution	LCDW=80 , LCDH=60 (downscaled via compressimage)	Full native (e.g. 376×240); no downscale	60×80 or 120×188 depending on module
Camera buffer	mt9v03x_image[120][188] (DMA target)	mt9v03x_csi_image (pointer, toggles between csi1_image and csi2_image)	mt9v03x_image
Binary image	Pixel[LCDH][LCDW] (Pixel[60][80])	img_thres_data[H][W] (AT_DTCM_SECTION_ALIGN)	Pixel[LCDH][LCDW]
Grayscale working	Image_Use[LCDH][LCDW] (Image_Use[60][80])	img_raw (image_t → mt9v03x_csi_image)	Image_Use[LCDH][LCDW]
Drawing buffer	img_line_data[188][120] (declared [w][H] ; see Caveats)	img_line_data[H][W] (AT_DTCM_SECTION_ALIGN)	—
Struct type	image_t (from segmentation.h)	image_t / fimage_t (from imgproc.h)	Raw arrays; no image_t
Pixel access	IMAGE_Pixel(img, x, y)	AT_IMAGE(img, x, y)	Manual indexing
Memory placement	#pragma location = 0x2808'070c (inter-core), #pragma location = 0x28026024 (DMA temp), __no_init	AT_DTCM_SECTION_ALIGN(..., 64) in NonCacheable DTCM	#pragma section all "cpu0_dsram"

Thresholding

- **CYT4BB7/RT1064:** Adaptive threshold via `findline_lefthand_adaptive` / `findline_righthand_adaptive` (local mean minus `clip_value`). Also supports fixed threshold and Otsu (`getOSTUThreshold`).
- **TC264:** Otsu's method with dynamic compensation (range 180–215, compensation ≈ -10).

Boundary tracking

- **CYT4BB7/RT1064:** Left/right hand adaptive maze tracking (`Lmaze_Adapt` / `Rmaze_Adapt` or `findline_lefthand_adaptive` / `findline_righthand_adaptive`), producing `ipts0` / `ipts1` arrays. Then perspective-remapped to `rpts0` / `rpts1` using `mapx` / `mapy` lookup tables (generated offline by `RUBO-UDIPM-main` or OpenCV).
- **TC264:** Per-row left/right boundaries stored in `ImageDealDatatypedef ImageDeal[LCDH]`.

Road types (TC264 enum `RoadType_e`)

`Normal`, `Straight`, `Cross`, `Ramp`, `LeftCirque`, `RightCirque`, `Forkin`, `Forkout`, `Barn_in`, `Barn_out`.

Image Data Structures & Memory Layout

`image_t` (shared between CYT4BB7 and RT1064)

The RT1064 reference code and the CYT4BB7 port use the same lightweight image descriptor:

```
typedef struct image {
    uint8_t *data;
    uint32_t width; // columns
    uint32_t height; // rows
    uint32_t step; // row stride in bytes (usually == width)
} image_t;
```

Key macros:

- `DEF_IMAGE(ptr, w, h)` $\rightarrow$ `{.data=ptr, .width=w, .height=h, .step=w}`
- `AT_IMAGE(img, x, y)` $\rightarrow$ `(img)->data[(y)*(img)->step+(x)]`
- `AT_IMAGE_CLIP(img, x, y)` $\rightarrow$ clamped border access
- `IMAGE_Pixel(img, x, y)` (CYT4BB7 naming) is equivalent to `AT_IMAGE`.

CYT4BB7 memory map

- `mt9v03x_image[MT9V03X_H][MT9V03X_W]` ([120][188]): Final DMA-received image in CM7_0.
- `mt9v03x_image_temp[MT9V03X_H][MT9V03X_W]`: DMA scratch buffer placed at `#pragma location = 0x28026024`, marked `__no_init`. Copied to `mt9v03x_image` in `camera_finish_callback()` with DCache invalidation (`SCB_InvalidateDCache_by_Addr`).
- `m7_1_data[MT9V03X_IMAGE_SIZE]` (22560 bytes): Inter-core communication image.
 - In CM7_1: `#pragma location = 0x2808'070c`.
 - In CM7_0: `__no_init uint8_t m7_1_data[MT9V03X_IMAGE_SIZE]`.

- `Image_Use[LCDH][LCDW] ([60][80])`: Downscaled grayscale image (nearest-neighbor from 188×120).
- `Pixel[LCDH][LCDW] ([60][80])`: Binary image after thresholding.
- `img_line_data`: Display/draw buffer. **Caveat:** declared as `uint8_t img_line_data[MT9V03X_W][MT9V03X_H] ([188][120])` but used with `image_t` expecting row-stride 188. Because C row-major layout of `[188][120]` has row-size 120, this is dimensionally transposed. The code works only because it is always accessed through the `image_t` pointer and never via 2D indexing.

RT1064 memory map

All performance-critical image and point arrays are forced into **DTCM** (Data Tightly Coupled Memory, non-cacheable) with 64-byte (or 8-byte) alignment:

```
AT_DTCM_SECTION_ALIGN(uint8_t mt9v03x_csi1_image[MT9V03X_CSI_H][MT9V03X_CSI_W],
64);
AT_DTCM_SECTION_ALIGN(uint8_t mt9v03x_csi2_image[MT9V03X_CSI_H][MT9V03X_CSI_W],
64);
AT_DTCM_SECTION_ALIGN(uint8_t img_thres_data[MT9V03X_CSI_H][MT9V03X_CSI_W], 64);
AT_DTCM_SECTION_ALIGN(uint8_t img_line_data[MT9V03X_CSI_H][MT9V03X_CSI_W], 64);
```

The CSI driver toggles `mt9v03x_csi_image` between `csi1_image` and `csi2_image` for zero-copy double buffering.

TC264 memory map

Image buffers are placed in CPU0 DSRAM via `#pragma section all "cpu0_dsram"` for fast zero-wait-state access. Boundary clamping macros: `CLIP_DATA(x)` clamps to row range, `CLAMP(x)` clamps to column range.

Control System

Multiple independent PID controllers:

- **Speed loop** (`speed_velocity()`): $K_P = -0.035$, $K_I = -0.0018$
- **Angle loop** (`angle_velocity()`): $K_P = 720$, $K_D = 1$
- **Turn loop** (`turn_pid()`): $K_D = 0.22$ with gyro integration
- **Gyro speed** (`gyro_pid_velocity()`): $K_P = 0.79$

RT1064 reference adds: Pure-pursuit steering (`atanf(pixel_per_meter * 2 * 0.2 * dx / dn / dn)`) combined with PID servo control (`servo_pid`). Dynamic pre-aim distances (`aim_distance ≈ 0.58m`, near aim `0.25m`).

Sensor Fusion

- **Camera:** MT9V03X (primary), OV7725 (alternative)
- **IMU:** IMU660RA / ICM20602 (Pitch/Roll/Yaw via quaternion or direct integration)
- **Encoders:** Motor speed feedback
- **TOF:** Distance-based obstacle detection
- **OpenART mini:** Apriltag / black-spot detection (RT1064 reference)

Tasking / Interrupt Architecture

CYT4BB7 (dual-core Cortex-M7)

- **CM7_0** (`project/user/main_cm7_0.c`): Main loop at 100 ms. Initializes camera (`mt9v03x_init`), IMU (`imu660rb_init`), servos, display. Displays raw `mt9v03x_image` when `mt9v03x_finish_flag == 1`. Can copy to `m7_1_data` for CM7_1.
- **CM7_1** (`project/user/main_cm7_1.c`): Main loop at 300 ms. Reads `m7_1_data` when `PROCE_FLAG == 0`, runs `compressimage()`, displays `Image_Use`, then sets `PROCE_FLAG = 1`. Uses `SCB_CleanInvalidatedDCache_by_Addr` for cache coherency.
- **PIT timers**: `PIT_CH0` 1 ms (balance control), `PIT_CH1` 10 ms (key scan).

RT1064 (RT-Thread + ISR)

- **CSI VSYNC ISR** (`isr.c`): Signals `camera_sem` when frame complete; toggles `mt9v03x_csi_image` pointer.
- **Main task** (`main.c`): Blocks on `rt_sem_take(camera_sem)`, then runs `process_image()` and `find_corners()`.
- **Debugger task**: `debugger_worker()` sends images/parameters to the SeekFree assistant上位机 when `DEBUGGER_PIN` is high.

TC264

- `CCU6_0_CH0` ISR (`isr.c`): Main periodic callback — image processing, control loop.
- `CCU6_0_CH1` ISR: Secondary timing — special maneuvers (obstacle jump, bridge crossing).
- TC264 does not support interrupt nesting by default; `interrupt_global_enable(0)` enables it.

Key Code Files

CYT4BB7

File	Purpose
<code>project/user/main_cm7_0.c</code>	CM7_0 main loop: peripherals, camera acquisition, raw display, inter-core copy
<code>project/user/main_cm7_1.c</code>	CM7_1 main loop: image processing, <code>compressimage</code> , display <code>Image_Use</code> , <code>PROCE_FLAG</code> sync
<code>project/code/segmentation.h/c</code>	<code>image_t</code> struct, <code>DEF_IMAGE</code> , <code>IMAGE_Pixel</code> , clone/clear, left/right adaptive maze tracking (<code>Lmaze_Adapt</code> / <code>Rmaze_Adapt</code>), point-set filters (<code>blur_points</code> , <code>resample_points</code> , <code>local_angle_points</code> , <code>nms_angle</code>)

File	Purpose
<code>project/code/pretreatment.h/c</code>	High-level pipeline: <code>FindEdge</code> (starting-point search + adaptive maze), <code>FindCorner</code> (WIP), undistort/IPM hooks
<code>libraries/zf_device/zf_device_mt9v03x.c/h</code>	MT9V03X SCCB/DMA driver, <code>mt9v03x_image</code> , <code>mt9v03x_image_temp</code> at fixed RAM addresses
<code>libraries/zf_driver/zf_driver_dma.c</code>	DMA configuration for camera data transfer

RT1064 (SJTU Reference)

File	Purpose
<code>Project/USER/src/main.c</code>	RT-Thread main task: camera sem wait, <code>process_image</code> , <code>find_corners</code> , garage/cross/yroad/circle/apriltag state machines, pure-pursuit servo control, debugger output
<code>Project/CODE/imgproc.h/c</code>	<code>image_t</code> / <code>fimage_t</code> , <code>AT_IMAGE</code> , threshold/adaptive-threshold, morphological ops, <code>findline_leftthand_adaptive</code> , <code>findline_righthand_adaptive</code> , point-set geometry filters
<code>Project/CODE/{cross,yroad,circle,garage,apriltag}.c</code>	Road-element detection and drawing routines
<code>Project/CODE/camera_param.c</code>	Camera intrinsic <code>K[3][3]</code> , distortion <code>D[4]</code> , homography <code>H[3][3]</code> and <code>H_inv</code> for IPM
<code>Libraries/seekfree_peripheral/SEEKFREE_MT9V03X_CSI.c/h</code>	CSI camera driver with double-buffered <code>mt9v03x_csi1_image</code> / <code>mt9v03x_csi2_image</code>
<code>Libraries/seekfree_libraries/common/common.h</code>	<code>AT_DTCM_SECTION_ALIGN</code> , <code>AT_ITCM_SECTION_INIT</code> , <code>clip</code> , <code>MINMAX</code> macros

TC264 (from prior analysis; code currently in zip)

File	Purpose
<code>user/cpu0_main.c</code>	Main loop: display, debug info, image visualization
<code>user/isr.c</code>	Interrupt handlers: periodic control callbacks
<code>code/image.c/h</code>	Image acquisition, Otsu thresholding, binarization
<code>code/Tracking.c/h</code>	Lane boundary detection, road type classification
<code>code/pid.c/h</code>	PID controllers for speed, angle, turn

File	Purpose
code/Common.c/h	Utility macros (CLAMP, CLIP_DATA, MY_ABS), image filtering, curvature
code/Obstacle.c/h	Obstacle detection and avoidance
code/Circle.c/h	Circle/roundabout navigation
code/Zebra_Crossing.c/h	Zebra crossing detection
code/servo.c/h	Servo/motor PWM control
code/Menu.c/h	On-screen parameter tuning menu
code/quaternion.c/h	IMU quaternion processing
libraries/zf_driver/	Hardware abstraction (GPIO, SPI, UART, PWM, DMA)
libraries/zf_device/	Peripheral drivers (camera, display, IMU, WiFi, etc.)
libraries/zf_common/	Shared utilities and header includes

Code Conventions

- **Language:** C (embedded, no standard library reliance). Asserts are used in CYT4BB7/RT1064 (`assert(img && img->data)`).
- **Naming:** CamelCase for types/functions (`ImageDealDatatypedef`, `RoadType_e`), snake_case for variables (`speedring_KP`).
- **Display debug:** Heavy use of `ips200_show_*` calls for real-time parameter visualization on IPS200 screen.
- **Image buffers:** Placed in fast/local RAM via `#pragma location` (CYT4BB7), `AT_DTCM_SECTION_ALIGN` (RT1064), or `#pragma section att "cpu0_dsram"` (TC264).
- **Boundary clamping:** Use provided macros — `CLIP_DATA(x)` clamps to [0,59], `CLAMP(x)` to [0,79] (TC264); `clip(x, lower, upper)` (CYT4BB7/RT1064).
- **Comments:** Primarily in Chinese (GBK encoding — may display as mojibake in UTF-8 editors).
- **Caveat — array dimensions vs `image_t`:** When declaring a buffer for `DEF_IMAGE`, the C array should be `[height][width]` so that row-major stride equals `width`. In `project/code/segmentation.c` the buffer `img_line_data` is declared `[MT9V03X_W][MT9V03X_H]` (`[188][120]`), which yields a row size of `120` while `image_t.step` is set to `188`. Always access such buffers exclusively through the `image_t` pointer/macros to avoid indexing errors.

Subdirectories in vision/

- `八邻域_v2.0/` — **Core vision algorithm reference** (8-neighborhood connected-component boundary tracking). Contains `image.c.txt` and `image.h.txt` (full implementation for 188×120 grayscale → Otsu → 8-neighbor search → morphological filter), plus `智能车图像处理辅助文档v2.0.pdf` and Excel sheets. **Note:** source files use `.txt` extension.

- [RUBO-UDIPM-main/](#) — PC-side Qt tool for **undistortion and inverse perspective mapping (IPM)**. Used offline to generate [mapx / mapy](#) lookup tables and camera parameter files ([K](#), [D](#), [H](#)) consumed by the embedded firmware.
- [GN42A_Product-master/](#) — GN42A dual-frequency GPS module (CASIC TA1201, 115200 baud). **Not directly related to the vision pipeline.**
- [ipm/](#) — **Extensive image-processing reference archive.** Includes:
 - [NUC_01/](#) and [NUC_01国赛前/](#) — Infineon TC264 historical competition code (17th Smart Car).
 - [NUC_苏格拉_finally/](#) — Filter algorithm references ([filter.h](#), [LPfilter.c](#)).
 - [canny算法.c](#), [逆透视转c语言.txt](#), [摄像头矫正程序_yandeli.doc](#).
 - Multiple [.docx](#) guides: [坡道处理](#), [解决丢线问题](#), [速度决策](#), [动态前瞻使用方法](#).
- [CYT4BB7相关例程/](#) — SeekFree motherboard official demos (v5.3). Contains [CYT4BB7/](#) (active target libraries), [CYT4BB7_Library-master-5.3updated](#), and [TC264-Motherboard_Demo-5.3](#).
- [第十七届智能车RT-Thread群体挑战赛WIFI通讯例程/](#) — 17th Smart Car RT-Thread WiFi communication demo (MM32F327X). Not part of the current vision mainline.
- [【文档资料】摄像头相关资料/](#) — Camera documentation, datasheets, and calibration materials.